Welcome to Systems Fundamentals!!!

COMP211 Connor McMahon

Who am I?

- Professor Connor (“Cece”) McMahon
  - I usually go by my nickname
  - My last name is pronounced “McMan” – it’s Irish 🤦‍♀️
- This is my second year at UNC and first time teaching 211! 🎉
- Previously at UC Berkeley and Georgia Tech

TA Team

- Ben Chesser
- Lauren Feldman
- Martim Gaspar
- Martha-Grace Jackson
- Aashvi Jain
- Aum Kendapadi
- Avi Kumar
- Alex Marum
- Mira Mohan
- Akhil Motiramani
- Akshan Sameullah
- Ray Shealy
- Nahum Yared

Systems Fundamentals

- This course is designed around three content areas
  1. Systems Programming
  2. Representation Translation
  3. Tools for Computer Scientists

Application vs Systems Programming

- Application Programming
  - Produces software that directly serves the user
  - Examples: Email, Web Browsers, Word Processors (Microsoft Word), Video Conferencing
- Systems Programming
  - Produces software that provides an interface between the hardware and the programmer or end-user
  - Application software runs on top of system software
  - Operating Systems
    - Linux, Windows, MacOS
  - Device drivers
    - Software that enables your computer to communicate with hardware devices like keyboards, printers, or graphics cards
  - Compilers
    - Translate programs into a form that computers can understand and execute

Why Learn Systems Programming?

- Understand how computers work
- Optimizing software performance
  - Understand how to optimize for performance and scalability
  - Become adept at working with resource-constrained environments
- Improved debugging and problem-solving skills
  - Helpful to have under-the-hood knowledge
  - Learn to design software that interacts predictably with other software and hardware
- Building foundations for advanced topics
  - Operating Systems
  - Compilers
  - Networking
  - Cybersecurity

Why Learn Systems Programming?

- Careers that heavily rely on systems programming
  - Embedded systems, cybersecurity, game engine development, and cloud computing
- How is this course useful to software engineers?
  - Understand how computers work
  - Optimizing software performance
  - Improved debugging and problem-solving skills

The C Programming Language

- We'll be learning the C Programming Language
- Developing software in C can be more challenging compared to other languages. So why learn it?
  - C provides much finer control over the underlying hardware, making it indispensable for certain applications
  - This low-level control allows C to achieve significantly higher efficiency compared to higher-level languages like Java or Python.
- To illustrate C's performance benefits, we'll look at an example from the paper *There's Plenty of Room at the Top*

Performance Engineering

- The following Python program multiplies two 4096-by-4096 matrices

```
for i in xrange(4096):
    for j in xrange(4096):
        for k in xrange(4096):
            C[i][j] += A[i][k] * B[k][j]
```

On a modern computer (2020), it takes 25,552.48 seconds (around 7 hours) to execute this program.

If we implement the exact same program in Java or C, we see dramatic performance improvements.

https://www.science.org/doi/10.1126/science.aam9744

Performance Engineering

```
for i in xrange(4096):
    for j in xrange(4096):
        for k in xrange(4096):
            C[i][j] += A[i][k] * B[k][j]
```

Implementation	Running Time (s)	Absolute Speedup
Python	25,552.48	1
Java	2,372.68	11
C	542.67	47

https://www.science.org/doi/10.1126/science.aam9744

What causes this performance difference?

- Interpretation vs Compilation
  - Python
    - Python is an interpreted language, meaning your code is executed line-by-line by the Python interpreter, which adds overhead.
  - C and Java
    - C is a compiled language, meaning that it is translated into machine code before execution.
    - Java is compiled into bytecode, which is interpreted by the Java Virtual Machine (JVM). The interpretation layer makes Java slower than C.
- We'll learn more about these concepts throughout the course!

What causes this performance difference?

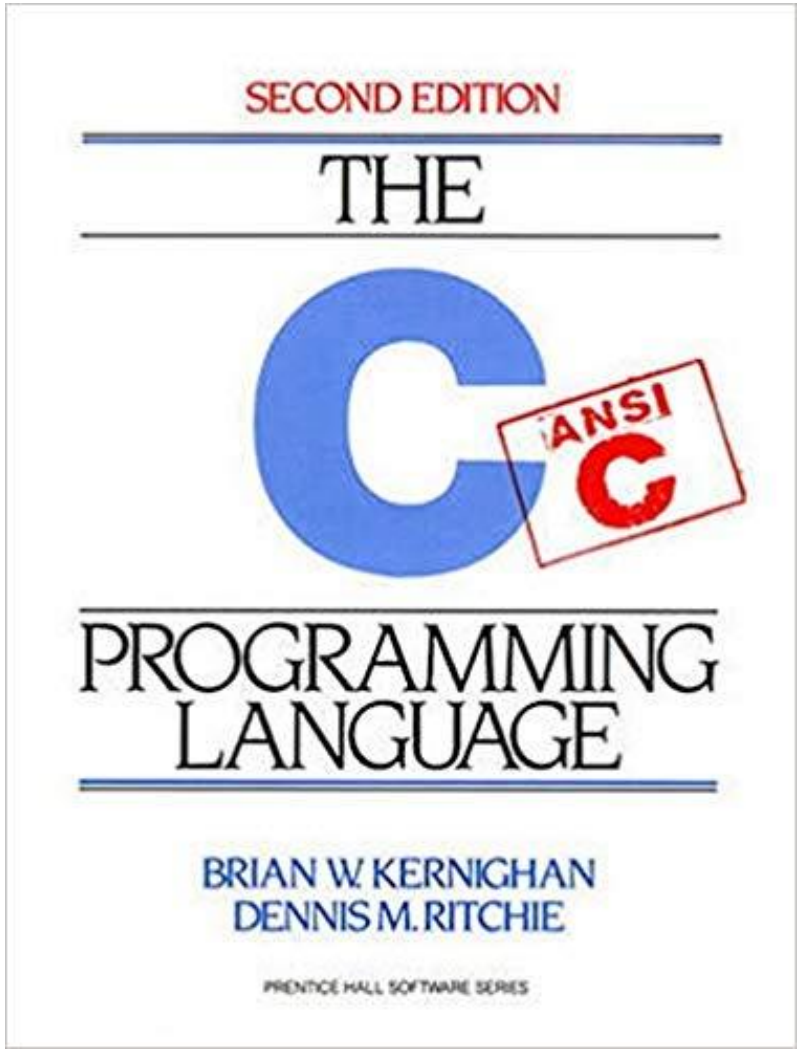
- Memory Management
  - Python and Java
    - Automatic memory management: the programmer does not have to worry about managing memory, but adds significant overhead
  - C
    - Manual memory management: you are in direct control of your program's memory and can manage it much more efficiently
- We'll learn more about these concepts throughout the course!

Syllabus

Please see the syllabus on the course website for the most up-to-date information

Textbook

- Written by Brian Kernighan and Dennis Ritchie
- A CLASSIC book in the field
- Available at Student Stores or on Amazon
- REQUIRED readings from this text begin next week



Grading Criteria

- 40% - Preparation, Practice, Participation
  - 10% - (HW) Homework
  - 25% - (LAB) Lab Exercises
  - 5% - (CL) In-class Participation (Graded for Completion)
- 60% - Mastery
  - 45% - 3x Exams
  - 15% - Final Exam

Homework and Labs

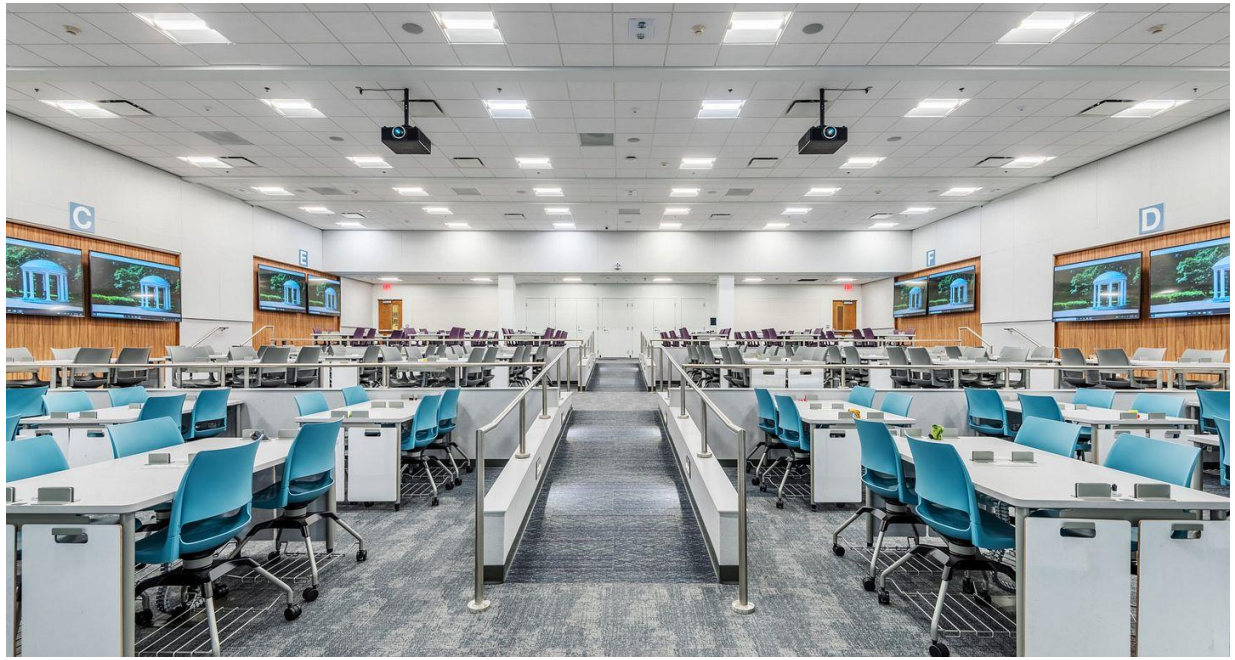
- Autograded with immediate feedback on Gradescope
- Assignments must be completed individually
- ChatGPT is not allowed
- No regrade requests
- No drops

Homework and Labs – Slip tokens

- All students are given eight slip tokens at the beginning of the semester
- One slip token can be used to submit a homework or lab assignment one day late without penalty
- A maximum of four slip tokens may be used on a single assignment
- Any assignment submitted more than four days after the due date will receive a 0
- Any assignment submitted late after all slip tokens have been used will receive a 0

Active Learning - Participation

- Active learning activities improve learning outcomes
- You are expected to work with your peers on the active learning questions
- You'll submit your participation activity during each class (graded for completeness)



Section 2: Carroll 111

Assessments

- 3 Mid-semester Exams and a Final Exam
- Format
  - paper
  - closed-book, closed-note
  - Mid-semester Exams: 50 minutes
  - Final: 3 hours
- Assessment dates
  - Exam dates will be announced two weeks before exam
  - Final Exam
    - Section 1: Monday, May 5 @4pm
    - Section 2: Thursday, May 1 @4pm
- Scope
  - Exams: All material up to one week before the exam
    - e.g. If there is an exam on 10/28, the scope will be all content up to and including 10/21
  - Final exam: Cumulative
- Grading
  - Regrade requests must be submitted within 72 hours of grade release

Exam Absences

- Make-up exams will only be given to students who have a university-approved absence

Final Exam Replacement Policy

- If you take all 3 exams, the final exam will replace your lowest exam score
  - assuming the final exam score is higher than your lowest exam score
- If you do not take all 3 exams, you are not eligible for final exam replacement

Grading

- A: 93-100
- A-: 90-92
- B+: 87-89
- B: 83-86
- B-: 80-82
- C+: 77-79
- C: 73-76
- C-: 70-72
- D: 60-69
- F: 59 or below

Honor Code

- No posting assignments on github or other public websites
- See syllabus for full honor code policy

Slides

- Slides are posted on Canvas before class
- After class, slides with solutions to the active learning problems are posted

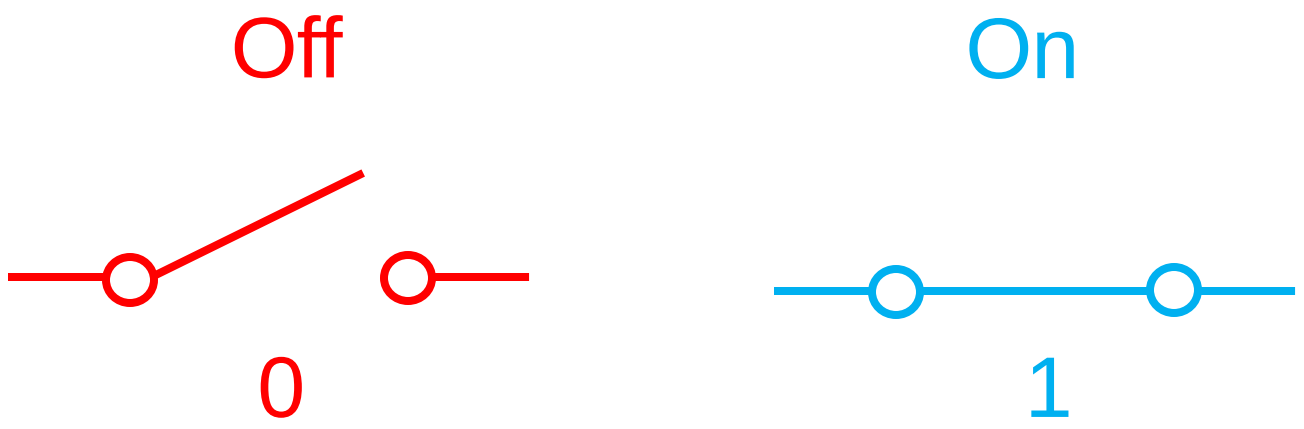
Action Items

- Command Line Interface Reading
  - Reading can be found on Canvas under Files -> Readings
  - Complete Part 0 before class on Monday, January 13
  - Gradescope assignment for Parts 1 and 2 is due on Wednesday, January 15

Binary

Binary

- The **binary number system** is a method for representing numbers with two symbols, usually “0” and “1”.
- Data in computers is represented using the binary number system.
- Why? The fundamental building block of computers is a **transistor**. A transistors behaves as a switch that can either be in an “on” state or an “off” state. You’ll learn more about transistors in COMP 311!



One transistor can represent two numbers: 0 and 1

Base

- A **base** is the number of different digits that a system can use to represent numbers
- Binary is **base-2**
  - It uses the digits 0 and 1
- Decimal (the system of representing numbers that we are used to) is **base-10**
  - It uses the digits 0 through 9

Binary

Decimal	Binary
0	
1	
2	
3	

Binary

- When writing a number in binary, we always need to indicate that it is a binary number. One way to do this is by prepending the number with **0b**.
- Each digit of a binary number is called a **bit**. A bit is a digit that can represent two values: 0 or 1
  - In the table on the right, each binary number is composed of two bits.

Decimal	Binary
0	0b 00
1	0b 01
2	0b 10
3	0b 11

If we did not prepend this value with a 0b, we would assume that it is the decimal number ten! Make sure you always remember to prepend with 0b!

Binary

- With two bits, we can represent four unique values ($2^2 = 4$).

Decimal	Binary
0	0b 00
1	0b 01
2	0b 10
3	0b 11
4	
5	
6	
7	

Binary

- With two bits, we can represent four unique values ($2^2 = 4$).
- With three bits, we can represent eight unique values ($2^3 = 8$).
- Notice that we added one “0” to the front of the binary values that correspond to decimal numbers 0-3.

Decimal	Binary
0	0b 000
1	0b 001
2	0b 010
3	0b 011
4	0b 100
5	0b 101
6	0b 110
7	0b 111

Active Learning Question

- With four bits, we can represent 16 unique values (2^4 =16).
- Fill in the remainder of the table.
- Poll Everywhere: What is the 4-bit binary representation of 12?
  - 0b 1100

pollev.com/comp211

Decimal	Binary
0	0b 0000
1	0b 0001
2	0b 0010
3	0b 0011
4	0b 0100
5	0b 0101
6	0b 0110
7	0b 0111
8	
9	
10	
11	
12	
13	
14	
15	

Active Learning Questions

- What is the range unsigned of values that can be represented with n bits?

pollev.com/comp211

Binary

- There are two ways to indicate that a number is in binary representation
  - Prepend 0b
    - Ex: 0b1001
  - Subscript 2
    - Ex: 1001_2
- Place binary numbers in groups of four to make it easier to read
  - Normally, spaces are used when handwritten
    - Ex: 0b1010 1111 0101
  - And underscores are used for typed values
    - Ex: 0b1010_1111_0101
  - If you cannot evenly break up the number into groups of four, start making the groups from the right
    - Ex: 0b10_1010_0011

Binary Bit Positions

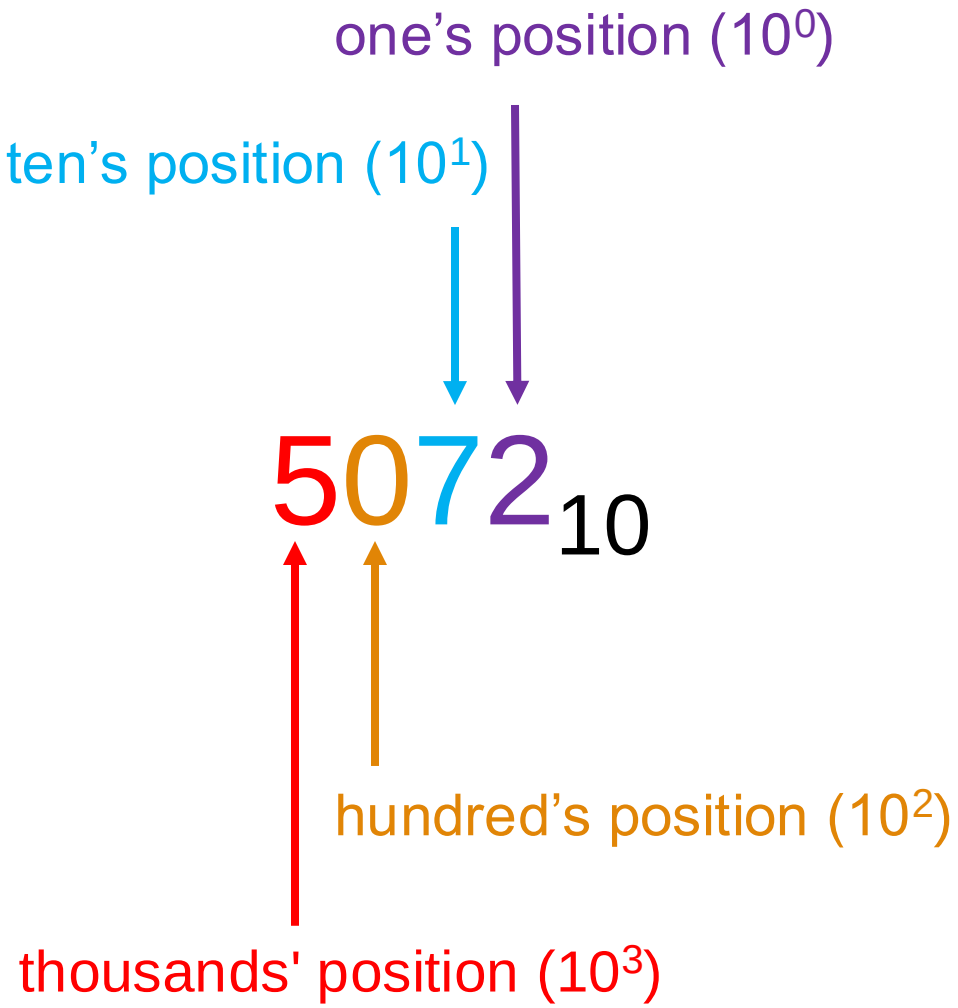
- We'll often want to reference individual digits of a binary number
- To do this, we need to give a name to each bit
- The bits are named n-1 to 0 where n is the number of bits in the number

$$1\ 0\ 1\ 1_2$$

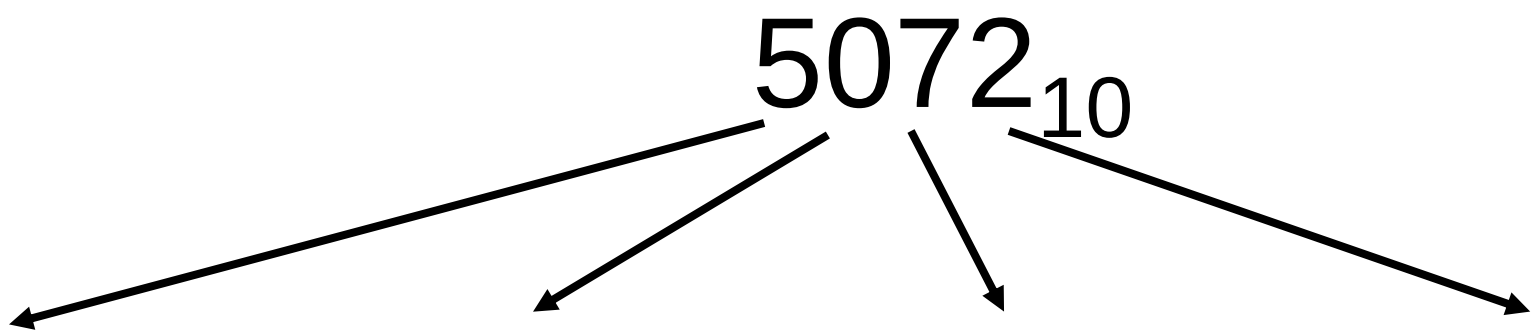
Bit 3 Bit 2 Bit 1 Bit 0

Base-10

Decimal Notation



Decimal Notation



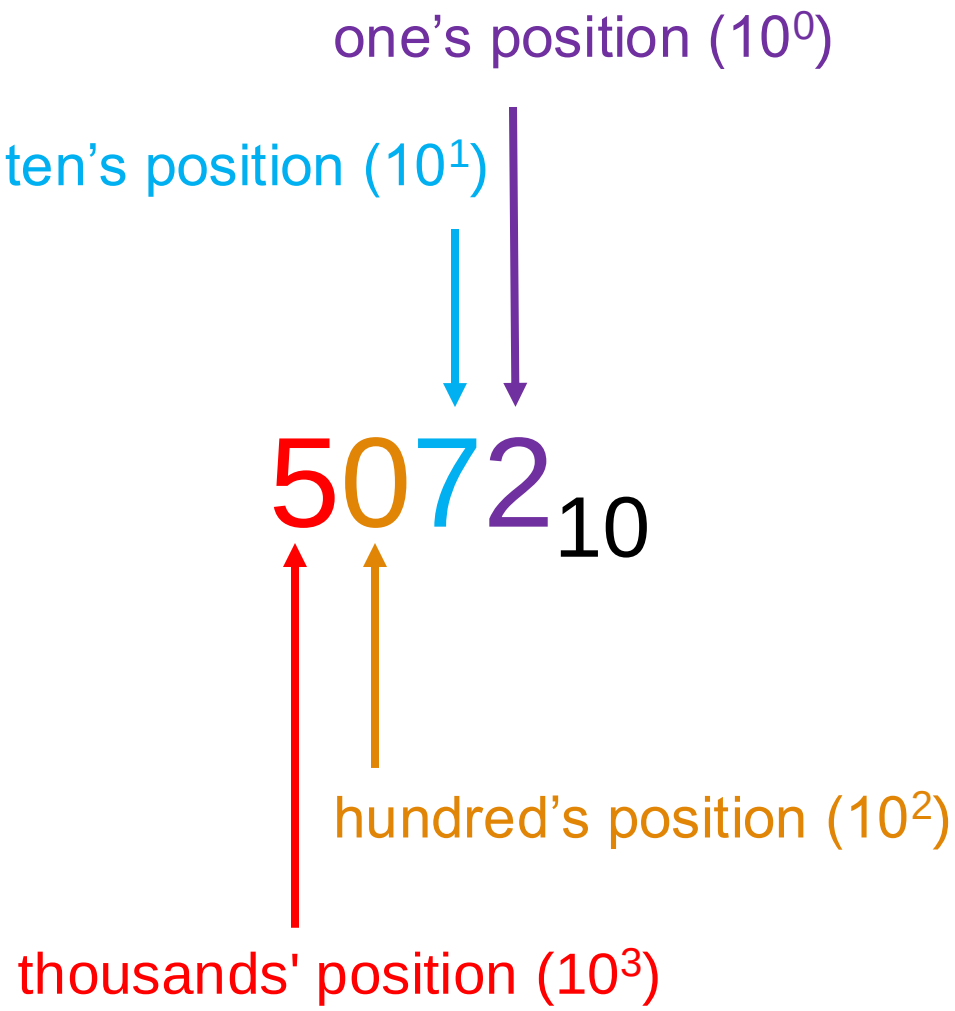
$$(5 \times 10^3) + (0 \times 10^2) + (7 \times 10^1) + (2 \times 10^0)$$

$$5000 + 0 + 70 + 2$$

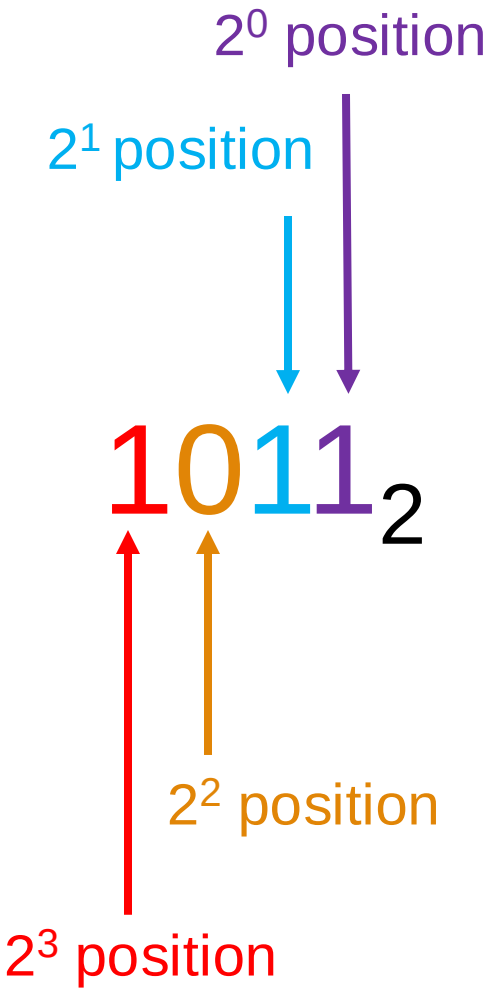
$$5072_{10}$$

Base-2

Decimal



Binary

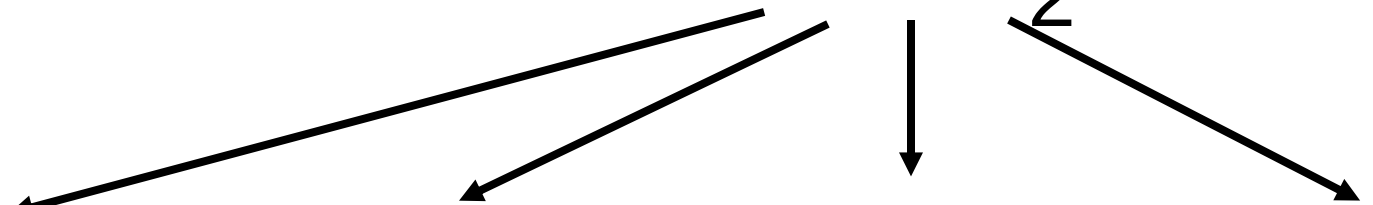


Convert from Binary to Decimal

$$1011_2$$

Convert from Binary to Decimal

$$1011_2$$



$$(1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$$

$$8 + 0 + 2 + 1$$

$$11_{10}$$

Converting from Binary to Decimal – your turn!

Q1. 0b 0010 0011

Q2. 0b 0101 0100

pollev.com/comp211